智慧图书馆数字资源平台

数据库设计与实现项目文档

MongoDB 7.0 · 单机 → 三节点副本集 → 两分片集群

项目名称	智慧图书馆数字资源平台
组队方式	单人组队
数据库	MongoDB 7.0.43（mongosh 2.12.0 / pymongo 4.19）
集群环境	NoSQL01 / NoSQL02 / NoSQL03（CentOS Stream 10）
姓　名	郭博涵
学　号	202414870301
完成日期	2026 年 11 月 10 日

一、项目概述

1.1 项目背景

高校图书馆的数字资源正在从“纸本书目 + 人工台账”转向“全量在线、可分析、可扩展”的服务平台。本项目以一套图书、读者、借阅流水与审计日志数据为基础，构建智慧图书馆数字资源平台的数据库底座，覆盖书目检索、借阅管理、统计分析、电子书与封面存储、高可用与水平扩展等完整链路。业务上要求平台既能支撑日常的“按类别、价格、标签、年份、评分”多条件检索，也能支撑管理侧的馆藏结构分析、学院借阅排行、借阅趋势与热门图书排行，并保证所有写操作留痕可审计。

1.2 建设目标

- 以文档模型对书目、读者、借阅流水三类主体建模，用一次查询取回业务所需的完整对象，避免多表连接。
- 为高频检索与聚合设计二级索引，用执行计划（explain）量化索引前后的差异，而不是凭感觉加索引。
- 用 Python + pymongo 实现可运行的业务应用，统一连接模块、统一审计写入，展示菜单化与脚本化两种使用方式。
- 用 GridFS 承载超过 16MB 限制的 PDF 电子书与封面图，并把 ISBN、类型、原文件名等元数据写回文件文档。
- 完成从单机到三节点副本集、再到“配置服务器 + 两分片 + 路由”的架构演进，验证故障转移、读取偏好与数据分布。

1.3 运行环境

项目	配置
宿主机	Windows + VMware Workstation Pro，VMnet8 NAT 网段 192.168.121.0/24
虚拟机	NoSQL01（.134）、NoSQL02（.135）、NoSQL03（.136），各 2 vCPU / 3.6GB 内存 / 45GB 磁盘
操作系统	CentOS Stream 10（内核 6.12），SELinux 设为 permissive，firewalld 关闭
数据库	MongoDB 7.0.43（mongod / mongos / mongosh 2.12.0）
应用	Python 3.12.14 + pymongo 4.19，代码部署于 /data/app

表 1-1 运行环境

1.4 数据与交付物

原始数据为三份制表符分隔的文本：books.txt（50 条书目，含 4 条无 tags）、readers.txt（30 条读者，含 2 条无 fav_categories）、borrow_logs.txt（3000 条借阅流水，已归还 2142 条、借出 429 条、逾期 429 条，跨度 2023—2025 年），另有 3 个 PDF 电子书与 3 张封面图用于 GridFS。交付物包括：可直接运行的 Python 源代码、数据库与配置（数据文件、备份、副本集与分片启动配置、端口说明）、本项目文档、34 张过程截图、8 分钟答辩 PPT。

二、技术选型

2.1 选型论证

本项目的核心数据是“书目 + 读者 + 借阅流水”。书目天然是半结构化的：不同类别的书需要不同的属性，计算机类图书关心标签（如 Python、算法、网络），文学类图书关心译者与丛书，外语类图书关心语种与等级；同一本书的馆藏位置（楼层、书架、复本数）是一个随馆舍调整而整体变化的子结构。如果采用关系型数据库，至少需要 books、book_tags、locations、readers、reader_fav_categories、borrow_logs 六张表，每次列表页查询都要 4～5 次 JOIN，标签的“同时包含多个标签”查询还要靠 INTERSECT 或多次 EXISTS 子查询，读写放大明显。文档模型把一本书的全部属性放在一个文档里，一次查询即可返回列表页需要的数据，多标签查询用 $all 一步完成，代价是需要在应用层保证跨文档的一致性。

从访问特征看，本平台是典型的“读多写少 + 聚合分析重”场景：检索与统计占请求的绝大多数，写入集中在借还书与后台导入。MongoDB 的二级索引、复合索引、多键（multikey）索引以及聚合管道，恰好覆

盖了“类别 + 价格区间”“多标签”“评分 Top10”“按年月分组”“学院维度汇总”这些查询；聚合管道还可以用 $lookup 在需要时做轻量关联，用 $out 把统计结果物化成新的集合，从而把重分析从在线查询中剥离出去。

在文件存储方面，电子书 PDF 单文件 37～40KB，虽未触及 16MB 的 BSON 上限，但平台未来会引入扫描版教材与随书光盘镜像，单个文件很容易超过上限；同时文件需要与书目元数据（ISBN）绑定、支持按书名检索、支持浏览器分段下载。把二进制直接塞进文档会撑大工作集、破坏文档的局部性，因此选择 GridFS：它把文件切成 255KB 的 chunk 存进 fs.chunks，元数据存进 fs.files，天然支持大于 16MB 的文件、支持按 metadata 建索引、支持流式读写。

高可用与扩展性方面，单机 MongoDB 一旦进程或磁盘故障，平台即不可用；而图书馆的借还服务属于“可以慢一点、但不能停”的业务。因此演进路径设计为：单机（功能验证）→ 三节点副本集（自动故障转移、读写分离）→ 两分片集群（按读者维度水平扩展）。副本集用 keyFile 做内部认证、用 w:majority 保证写入在多数派落盘，分片集群用 hashed 分片键把写入打散到多个分片，两者组合可以在不修改应用代码（只改连接串）的前提下完成升级，这也是选择 MongoDB 而非自建分库分表中间件的重要理由。

综上，选型结论是：以 MongoDB 作为主库承担在线事务与分析，以 GridFS 承担文件存储，以 Python + pymongo 作为应用层；不引入 Redis 做缓存（当前数据量与 QPS 不需要，且会带来缓存一致性成本），不引入关系型数据库做“双写”（会破坏单一事实来源）。

2.2 候选方案对比

对比维度	MongoDB 7.0	MySQL 8.0	Redis 7
数据模型	文档（BSON），字段可动态增减	关系表，需预定义 schema 与关联表	键值 / Hash / ZSet，无二级索引
多标签查询	$all 一步完成，多键索引加速	需标签表 + 多次 JOIN/EXISTS	需在应用层做集合运算
聚合分析	内置聚合管道，$group/$lookup/$out	SQL 聚合 + 物化视图	仅适合计数与排行
水平扩展	原生分片，按分片键自动路由	分库分表需中间件改造	Cluster 分片，但无强一致
事务能力	多文档事务（副本集/分片集）	强 ACID，成熟	无事务语义
文件存储	GridFS 原生支持大文件	需应用层分块 + BLOB	不适合存大文件
本项目适配度	高：半结构化 + 多条件检索 + 聚合 + 大文件	中：需 6 张表与大量 JOIN	低：不适合做主存储

表 2-1 主存储选型对比

存储方式	适用场景	优点	缺点	本项目结论
内嵌文档	1:1 或 1:少，且总是一起读取	一次查询取全，原子更新	文档膨胀、难以独立检索	采用：books.location
引用（手动关联）	1:N 且子文档会被独立查询/更新	避免重复、便于独立扩展	需要 $lookup 或两次查询	采用：borrow_logs 引用 readers/books
GridFS	>16MB 或需流式读写的二进制	突破 BSON 上限、可建元数据索引	多一次查询、chunk 开销	采用：PDF 与封面

表 2-2 数据组织方式对比

2.3 CAP 与 BASE 分析

按 CAP 理论，分布式系统在网络分区（P）发生时必须在一致性（C）与可用性（A）之间取舍。本项目部署在同一宿主机的三台虚拟机上，分区不是常态，但进程崩溃、主节点下台、网络抖动都会触发类似分区的效果。MongoDB 副本集在默认配置下（readPreference=primary、writeConcern=majority）表现为 CP：主节点不可用时，写请求会短暂失败，直到新的主节点被选举出来（本项目实测约 3.8 秒完成角色切换）。这一取舍符合图书馆业务：借书、还书、扣减库存必须读到最新数据，宁可让客户端重试几秒，也不能出现“超借”。

对于“馆藏浏览、排行榜、学院借阅统计”这类允许短暂陈旧的读请求，项目通过 readPreference=secondaryPreferred 把读流量导向从节点，此时系统在分区期间偏向 AP：从节点可能返回几秒前的数据，但服务不中断。也就是说，同一套集群通过读取偏好实现了“按业务分级的一致性”，而不是全局二选一。

BASE 思想体现在三处：其一，统计结果通过聚合管道定时物化到 category_stats 集合，允许最终一致，避免每次打开统计页都全表扫描；其二，借阅流水的 status 与 return_date 分两次写入（借出时写 status=

借出，归还时补 return_date），中间态由业务语义定义，不做分布式锁；其三，审计日志写入固定集合，采用“先写日志再返回”的顺序，即便极端情况下日志丢失一条，也不影响主业务数据。

2.4 选型结论与风险

结论：MongoDB + GridFS + Python 的组合能够以较低的实现成本覆盖本项目的建模、检索、聚合、文件存储、高可用与水平扩展需求，且架构演进只需修改连接串与部署形态，不改业务代码。

风险一：跨文档事务的缺失。借书操作需要同时扣减 books.stock、递增 books.borrow_count、插入 borrow_logs 并写 audit_logs，四个文档分属三个集合。项目的应对是：把库存扣减写成带条件的原子更新 updateOne({isbn, stock: {$gt: 0}}, {$inc: {stock: -1, borrow_count: 1}})，用返回的 matchedCount 判断是否成功，失败即中止后续写入；流水与审计的写入放在扣减成功之后，并用 try/except 记录补偿日志。这样在最坏情况下可能出现“库存已扣、流水未写”，但不会出现“库存为负”。若未来要求强一致，可在副本集上开启多文档事务。

风险二：分片键选择的长期影响。分片键一旦选定便难以更改（MongoDB 7.0 之前不可变，之后也只能通过 reshard 迁移）。本项目选择 hashed(reader_card)，写入分散、按读者查询可定向路由，但“按时间范围统计借阅趋势”会退化为广播到所有分片。如果未来业务重心转向时间维度的报表，应改为复合分片键 {reader_card: 1, borrow_date: 1} 或引入预聚合表，这一取舍已在第六章详细论证。

2.5 面向非技术决策者的一句话说明

我们用文档型数据库 MongoDB 替代传统多表数据库，把一本书的所有信息放在一条记录里，检索和统计更快、改版更灵活；文件（电子书、封面）单独存进数据库自带的文件仓库，随时可扩容；三台服务器互为备份，主服务器宕机约 4 秒自动接管，读者几乎无感知；后续读者量增长时，只需增加机器即可横向扩容，不用重写系统。

三、数据模型设计

3.1 逻辑结构

平台共有四个集合加一组 GridFS 文件：books 存书目，readers 存读者，borrow_logs 存借阅流水，audit_logs 存审计日志；电子书与封面以 GridFS 形式存放（fs.files / fs.chunks），通过 metadata.isbn 与 books.isbn 逻辑关联。borrow_logs 通过 reader_card 关联 readers、通过 book_isbn 关联 books，二者都是“引用”而非内嵌，因为流水数量会持续增长，且需要独立按时间、按读者、按图书三个维度查询。

关系	基数	实现方式	说明
books — borrow_logs	1 : N	borrow_logs.book_isbn → books.isbn	流水保留历史，不随书目删除
readers — borrow_logs	1 : N	borrow_logs.reader_card → readers.card_no	读者注销后流水仍保留
books — fs.files	1 : 0..2	fs.files.metadata.isbn → books.isbn	一本电子书 + 一张封面
books — location	1 : 1	内嵌子文档	楼层、书架、复本数一起读取

表 3-1 实体关系说明

3.2 字段与 BSON 类型

集合	字段	BSON 类型	示例 / 说明
books	isbn	string	9787115428028，业务主键，唯一索引
books	title / author / publisher	string	书名、作者、出版社
books	category	string	6 个类别之一：文学、计算机、数学、外语、经济管理、历史
books	price / rating	int / double	89、4.6
books	stock / borrow_count	int	库存与累计借阅次数
books	publish_date	date	ISODate，导入时由字符串转换
books	tags	array<string>	多键字段，如 ["编程","入门","Python"]
books	location	object	{floor, shelf, copies}
books	ebook_name	string	关联 GridFS 的文件名

集合	字段	BSON 类型	示例 / 说明
readers	card_no	string	R20240001，唯一索引
readers	name / college / level	string	姓名、学院、读者等级
readers	register_date	date	注册时间
readers	fav_categories	array<string>	偏好类别，部分读者缺失
borrow_logs	reader_card / book_isbn	string	外键引用
borrow_logs	borrow_date / return_date	date	借出、归还时间
borrow_logs	status	string	已归还 / 借出 / 逾期
borrow_logs	duration_days	int	借阅天数
audit_logs	ts / action / target / detail / operator / app	date / string	审计六要素

表 3-2 字段与 BSON 类型

3.3 易错字段与设计理由

（1）publish_date：源数据是 ISO8601 字符串（如 "2023-02-28"），若直接以字符串入库，$gte/$lt 会按字典序比较，“2023-1-5”会排在“2023-10-01”之后，年份区间查询必然出错。导入脚本统一用 datetime.fromisoformat 转成 BSON Date，并约定所有日期以 UTC 存储、展示时按 +08:00 转换，聚合中的 $dateToString 显式带 timezone。（2）price：价格必须用数值类型而非字符串，否则 $gte/$lte 无法做区间比较；同时要区分 int 与 double，本项目源数据为整数元，导入为 int，均价计算时由 $avg 自动升格为 double。（3）reader_card / isbn 这类业务主键必须建唯一索引，否则并发导入或重复提交会产生重复书目；MongoDB 的 _id 是 ObjectId，不具备业务含义，不能替代业务唯一约束。（4）status 用枚举字符串而不是布尔 is_returned：业务需要区分“借出”与“逾期”，布尔值无法表达三态；枚举在聚合分组时也更直观。

（5）tags 是数组，查询“同时包含 Python 与入门”必须用 $all 而不是 $in（$in 是“任一匹配”）；数组字段上的索引是 multikey 索引，一个文档会生成多个索引项，因此 $all 的扫描键数会大于返回文档数。（6）location 采用内嵌文档：楼层、书架、复本数只属于这本书，且列表页与详情页总是一起读取，内嵌后一次查询即可取全，更新时 $set 子字段也是单文档原子操作。若未来出现“按书架统计馆藏”“跨馆调拨”“书架本身有负责人与容量”等需求，就应拆成独立的 shelves 集合，用 shelf_id 引用，避免同一书架信息在成百上千个书目文档中重复。

3.4 集合类型选择

borrow_logs 选择普通集合。它需要支持“归还”操作（更新 return_date 与 status）、需要按三个维度建索引、也需要在分片集群中作为分片集合。固定集合（capped）不能修改文档大小、不能分片，时间序列集合虽然压缩率高，但对更新与分片键有额外限制（分片键只能是 metaField 或时间字段），都不满足借还业务。

audit_logs 选择固定集合（capped，size=32MB，max=200000）。审计日志只追加、不修改、不删除，按时间顺序读取，固定集合的顺序写与环形覆盖特性正好匹配；容量封顶后旧日志自动被覆盖，避免无限增长拖垮磁盘。代价是无法删除单条日志，这也符合审计“不可篡改”的要求。

3.5 索引设计

集合	索引	类型	服务的查询
books	uk_isbn	唯一	按 ISBN 精确定位、防止重复
books	ix_category_price	复合 (category, price)	类别 + 价格区间检索
books	ix_tags	多键 (tags)	$all / $in 多标签检索
books	ix_publish_date	单字段	出版年份区间
books	ix_rating_desc	单字段降序	评分 Top10
books	ix_borrow_count	单字段降序	热门图书排行
books	ix_title	单字段	书名前缀/模糊匹配（左前缀正则）
readers	uk_card_no	唯一	按卡号定位读者
readers	ix_college	单字段	按学院分组统计
borrow_logs	ix_reader_date	复合 (reader_card, borrow_date)	某读者借阅历史
borrow_logs	ix_book_date	复合 (book_isbn, borrow_date)	某书的借阅流水
borrow_logs	ix_borrow_date	单字段	按年月统计趋势
borrow_logs	ix_status	单字段	逾期清单

表 3-3 索引清单

3.6 关键设计决策

- D1 库名统一为 library_db，四个集合命名采用复数下划线风格（borrow_logs、audit_logs），与业务术语一致，便于运维排查。
- D2 书目与借阅流水分离存储：流水是只增不改的事实表，书目是会被修改的维表；分离后流水可独立分片、独立归档。
- D3 库存扣减用单文档原子更新 + 条件匹配（stock > 0），以数据库的原子性替代应用层锁，避免超借。
- D4 审计日志独立成固定集合，由应用统一封装 write_audit()，所有写操作（新增、删除、导入、导出、上传、借还）都必须留痕。
- D5 中文全文检索不建 text 索引：MongoDB 的 text 索引按空格分词，中文整句会被当作一个词，实际不可用；改为 ix_title 支持左前缀正则，并在文档中说明后续可接 Atlas Search 或外部分词服务。
- D6 统计结果物化：category_stats 由聚合管道 $out 生成，读统计页只查物化集合，把重计算从在线请求中剥离。

四、功能实现

4.1 数据导入与核对

导入脚本 tools/import_data.py 负责把三份文本导入数据库：逐行解析制表符分隔的字段，把日期字符串转为 BSON Date，数值字段转为 int/double，缺失的 tags、fav_categories 保持字段不存在（而不是写空数组），以保留“缺失”这一语义；随后用 insert_many 分批写入，再逐集合 countDocuments 与源文件行数比对，任何不一致立即报错退出。首次导入结果：books 50 / readers 30 / borrow_logs 3000，audit_logs 写入 1 条 IMPORT 记录，全部一致。

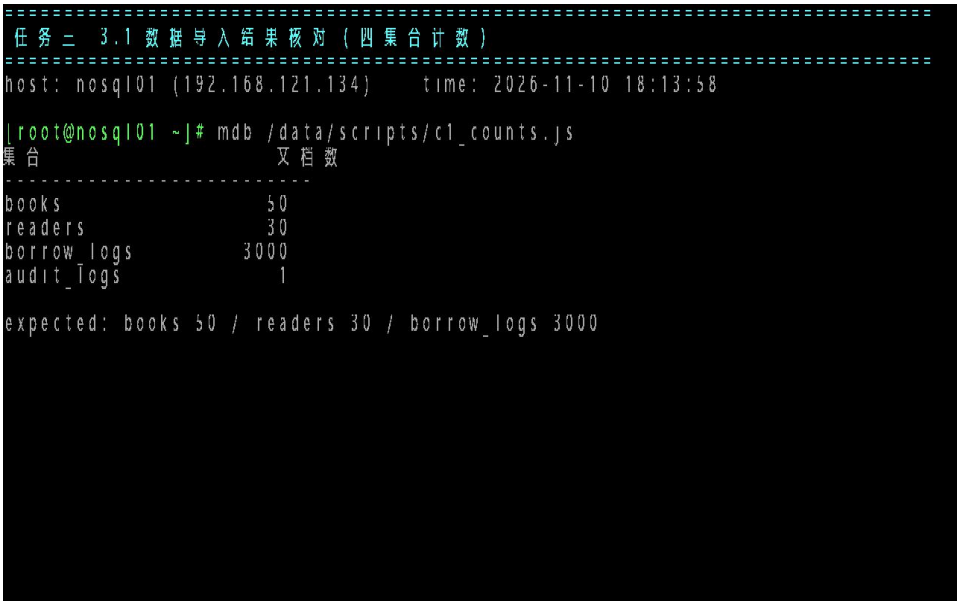


图 4-1 导入后四集合计数核对（books 50 / readers 30 / borrow_logs 3000）

4.2 检索功能

六类检索覆盖了文档数据库最典型的查询形态：

编号	需求	查询条件	命中
Q1	类别 + 价格区间	{category:"计算机", price:{$gte:80,$lte:150}}	7 条
Q2	多标签同时具备	{tags:{$all:["Python","入门"]}}	1 条
Q3	书名模糊匹配	{title:{$regex:"数据"}}	3 条
Q4	出版年份范围	{publish_date:{$gte:ISODate("2018-01-01"),$lt:ISODate("2021-01-01")}}	15 条
Q5	缺失 tags 字段	{tags:{$exists:false}}	4 条
Q6	评分 Top10（只返回三字段）	sort({rating:-1}).limit(10) + 投影	10 条

表 4-1 六类检索的实现与结果

4.3 统计分析

五项统计全部用聚合管道实现：按类别分组求藏书量与均价均分、按学院汇总借阅量并取前五、按年月统计借阅趋势、借阅最多的十本书并带出馆藏位置、以及把分类统计结果写入新集合 category_stats。其中“学院借阅 Top5”需要跨集合，用 $lookup 把 readers 的学院信息关联到流水上，再 $unwind 后按学院聚合；“按年月趋势”用 $dateToString 并显式指定 timezone: "+08:00"，避免 UTC 与本地时区错位导致跨月统计错误。

分类统计的结果为：文学 12 种 181 册居首，计算机 15 种 172 册次之，历史均价最高（95.60 元），具体数值见下表（截图 10）。

统计项	结果摘要
分类藏书量/均价/均分	文学 12 种 181 册 47.02 元 4.63 分；计算机 15 种 172 册 89.53 元 4.53 分；历史均价最高 95.60 元
学院借阅 Top5	计算机学院 500 次、经济管理学院 500 次、管理学院 400 次、历史文化学院 400 次、外国语学院 400 次
借阅趋势	按年月分组，2024 年为借阅高峰，与新生入学及课程周期吻合
热门图书 Top10	按 borrow_count 降序，同时输出 location.floor / shelf 便于找书
统计物化	category_stats 集合写入 6 条分类统计，供报表页直接读取

表 4-2 五项统计的结果摘要

4.4 索引优化与执行计划

索引的效果必须用执行计划量化。项目用同一段脚本在“建索引前”和“建索引后”执行两条典型查询，记录 winningPlan 的 stage、nReturned、totalKeysExamined、totalDocsExamined、executionTimeMillis，并做多次循环取总耗时。

查询	建索引前	建索引后	扫描文档数变化
A 类别+价格区间	COLLSCAN，扫描 50 条，2000 次共 858ms	IXSCAN(ix_category_price)，扫描 7 条，2000 次共 946ms	50 → 7
B 借阅月份区间	COLLSCAN，扫描 3000 条，500 次共 574ms	IXSCAN(ix_borrow_date)，扫描 85 条，500 次共 282ms	3000 → 85

表 4-3 正式库（3000 条）索引前后对比

值得注意的是查询 A 在建索引后反而略慢（858ms → 946ms）：books 集合只有 50 条文档，全表扫描本来就只有几十次比较，而走索引需要额外的索引查找与回表开销，数据量小时收益为负。为了真实体现索引价值，项目另外构造了 30 万条压测集合 bench_logs，用同样的查询对比：全表扫描平均 83.4ms/次（扫描 30 万个文档），建 ix_borrow_date 后平均 27.2ms/次（扫描 8509 个文档），提升约 3.1 倍，且扫描文档数与返回文档数完全一致，说明索引没有多余回表。压测结束后已删除该集合，正式库保持 3087 个对象、数据 35674.9KB、索引 10432.0KB 的干净状态。

执行计划显示，两条查询的阶段都由 COLLSCAN 变为 IXSCAN，扫描文档数分别从 50 降到 7、从 3000 降到 85，与索引键一一对应（截图 17）。

4.5 Python 应用

应用代码位于 /data/app/src，按职责分层：config.py 负责读取 config.ini 并支持环境变量覆盖，db.py 提供统一的 get_client / get_db / get_collection 连接入口（全局复用一个 MongoClient，设置 serverSelectionTimeoutMS=5000），audit.py 封装审计写入，books/readers/borrows/stats/files 各自封装领域操作，main.py 提供 12 项交互菜单与同名子命令。统一连接模块使“单机 → 副本集 → 分片集群”的切换只需改一行连接串，业务代码零改动。

借书功能体现了文档模型下的并发控制：先按 ISBN 与 stock>0 的条件原子扣减库存并递增借阅次数，再插入借阅流水、写审计；若库存不足则返回明确提示。还书功能按流水 _id 更新 status 与 return_date，并恢复库存。删除书目时先删除关联的 GridFS 文件、再删文档、最后写审计，顺序保证不会出现“文件孤儿”。

4.6 安全与权限

单机阶段即启用 authorization，建立三类账号：reader_ro（read，只读）、librarian_rw（readWrite，业务读写）、lib_admin（root，运维）。验证方式为正反两面：reader_ro 能读到 books 50 条、readers 30 条，

但 insert 与 drop 均被拒绝，返回 not authorized on library_db to execute command { insert }；librarian_rw 可以写入临时集合并清理，但 createUser 被拒绝；lib_admin 可以执行 serverStatus。副本集阶段用 keyFile 做节点间内部认证，多个节点必须使用同一份 keyFile，否则会出现 quorum check failed 与 Authentication failed。

五、文件存储设计

电子书与封面采用 GridFS 存储。GridFS 的机制是把文件按 255KB 切成若干 chunk 存入 fs.chunks，把文件名、长度、上传时间、以及自定义 metadata 存入 fs.files；读取时按 files_id 与 n 顺序拼接。这样做的好处是：突破单文档 16MB 上限、支持流式读写（不必把整个文件读进内存）、可以按 metadata 建索引实现“按 ISBN 找电子书”、并且与普通集合一样受副本集与分片集群的保护。

上传时写入的元数据包括 isbn（关联书目）、type（ebook / cover）、original_name（原始文件名）、size 与 content_type。项目对 3 个 PDF 与 3 张封面逐一做字节数校验：本地文件大小与 GridFS 中文件的 length 完全一致（39837 / 23611 / 38524 / 26053 / 37004 / 15171 字节），下载 ebook_1.pdf 后计算 md5 为 f818f1a754c2fabfdddf96f86294b9b4，与源文件一致，证明分块存储没有截断或错序。

文件	关联 ISBN	类型	字节数	校验结果
ebook_1.pdf	9787115428028	ebook	39837	一致，md5 f818f1a7…
ebook_2.pdf	9787121362726	ebook	38524	一致
ebook_3.pdf	9787115546081	ebook	37004	一致
cover_1.png	9787115428028	cover	23611	一致
cover_2.png	9787121362726	cover	26053	一致
cover_3.png	9787115546081	cover	15171	一致

表 5-1 GridFS 文件与校验结果

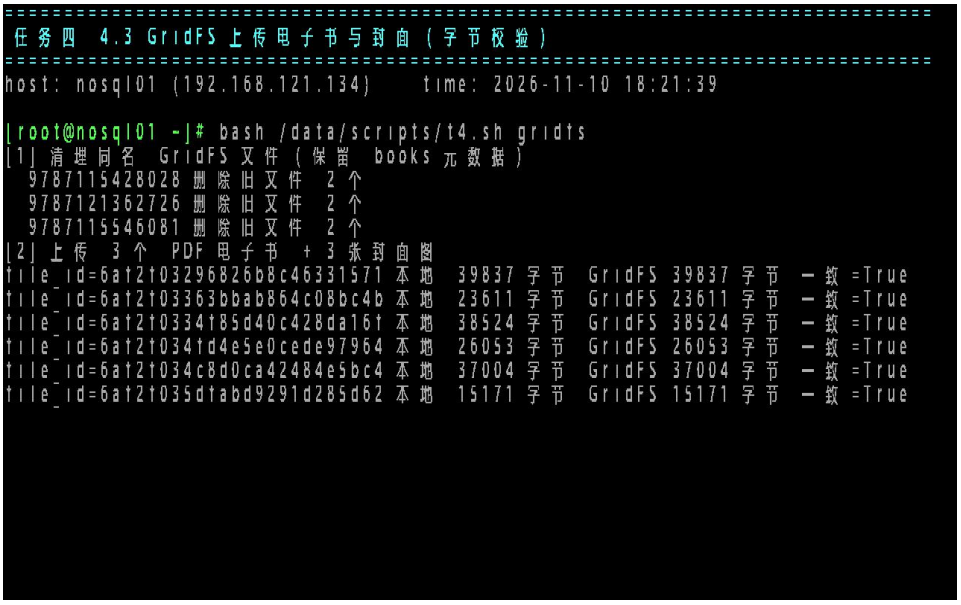


图 5-1 上传 6 个文件并逐一比对字节数，全部一致

六、架构演进

6.1 阶段一：单机实例

单机实例使用自定义数据目录 /data/mongodb/single/db，配置 wiredTiger.cacheSizeGB=0.5（受虚拟机内存限制）、端口 27017、bindIp 0.0.0.0、日志按追加方式写入文件、fork 方式后台运行。此阶段完成数据导入、检索、统计、索引优化与权限验证，是后续所有演进的功能基线。

6.2 阶段二：三节点副本集

副本集 rs0 由三台虚拟机构成，每台使用独立的 /data/mongodb/rs/db，端口仍为 27017，配置 replSetName=rs0 与 keyFile 内部认证，并启用 authorization。初始化后 192.168.121.134 为主节点，.135 与 .136 为从节点。副本集带来三项能力：自动故障转移、读写分离、以及 w:majority 的写入持久性。

读取偏好方面，项目验证了三种模式：primary 固定读主节点；secondaryPreferred 优先读从节点（主节点不可用时回落）；secondary 只读从节点（主节点故障时直接报错）。实测三次连接的 me 字段分别落在 .136（primary=true）、.135（secondary=true）、.134（secondary=true），证明读取偏好确实改变了请求落点。

故障转移演练用 replSetStepDown 让主节点下台，应用侧轮询感知新主节点耗时 3.8 秒，期间读请求短暂失败后自动恢复。更严苛的测试是在持续写入过程中让主节点下台：12 次写入中 11 次成功，第 9 次抛出 ServerSelectionTimeoutError（8008.4ms），第 10 次在 2274.5ms 后于新主节点成功，随后恢复正常。这说明 pymongo 的可重试写入能覆盖大部分切换场景，但应用仍需为“选举窗口内的失败”准备重试与提示逻辑。

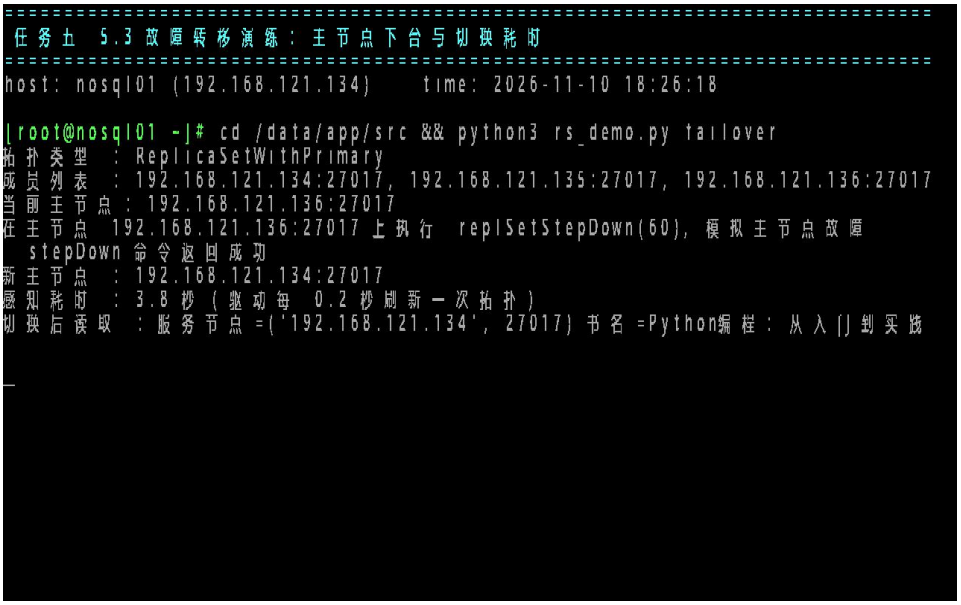


图 6-1 故障转移：新主节点在 3.8 秒内接管

节点增删演练先在从节点上执行 rs.remove 摘除一个成员，集群降为两成员（仍满足多数派），再用 rs.add 把它加回并等待追平。两次操作均返回 ok=1，最终恢复一主两从。该演练验证了扩容与缩容的运维路径，也暴露了一个约束：replSetReconfig 只能在可写的主节点上执行，客户端必须连接副本集地址而不是固定某一台。

6.3 阶段三：两分片集群

分片集群由三部分组成：配置服务器 csrs（三台虚拟机各一个，端口 27019，自身是一个副本集，保存集群元数据）、两个分片 shard1 与 shard2（分别位于 NoSQL01 与 NoSQL02，端口 27018，各自是一个副本集）、以及路由 mongos（位于 NoSQL03，端口 27020，对外提供统一入口）。应用只需连接 mongos，无需感知分片的存在。

组件	部署位置	端口	角色
config server csrs	NoSQL01/02/03	27019	保存 chunk 分布与会话元数据
shard1	NoSQL01	27018	分片 1（副本集）
shard2	NoSQL02	27018	分片 2（副本集）
mongos	NoSQL03	27020	查询路由与结果归并

表 6-1 分片集群组件与端口

6.4 分片键论证

分片键决定数据如何分布、查询能否定向。本项目的核心分片集合是 borrow_logs（3000 条，且会持续增长），候选方案有四种：其一，{borrow_date: 1}（范围分片）。借阅流水按时间单调递增写入，新数据永远落在最后一个 chunk，形成典型的“单调递增热点”：只有最后一个分片承受写入压力，其他分片闲置，水平扩展形同虚设；优点是按时间范围查询可定向路由，报表友好。其二，{book_isbn: 1}。ISBN 基数约 50，取值高度集中，热门书的流水会全部涌向同一个 chunk，分布严重倾斜，且无法解决热点问题。其三，复合键 {reader_card: 1, borrow_date: 1}。按读者范围分片，同一读者的流水集中在一个 chunk，写入热点取决于读者的活跃分布，30 位读者支撑 3000 条流水时，若干活跃读者的 chunk 会成为热点；且范围分片下 chunk

数量增长慢，均衡器迁移粒度粗。其四，{reader_card: "hashed"}。对读者卡号做哈希后取模，3000 条流水被打散到两个分片，实测 shard1 承载 1300 条（2 个 chunk）、shard2 承载 1700 条（2 个 chunk），分布接近均匀；写入压力随读者数增长而自然分散，不存在单调热点；代价是按读者查询可定向路由（SINGLE_SHARD），而按时间、按状态等非分片键条件的查询必须广播到所有分片后再归并（SHARD_MERGE）。考虑到图书馆的实际访问模式是“读者查自己的借阅记录”远多于“按时间段扫描全量流水”，且报表类查询可以走物化集合，本项目最终选择 hashed(reader_card)。实测对比：跨分片查询 {status:"逾期"} 走 SHARD_MERGE、涉及 2 个分片、返回 429 条、耗时 8ms；定向查询 {reader_card:"R20240001"} 走 SINGLE_SHARD、只涉及 1 个分片、返回 100 条、耗时 1～2ms，差了一个数量级的路由开销。

6.5 分片集群验收

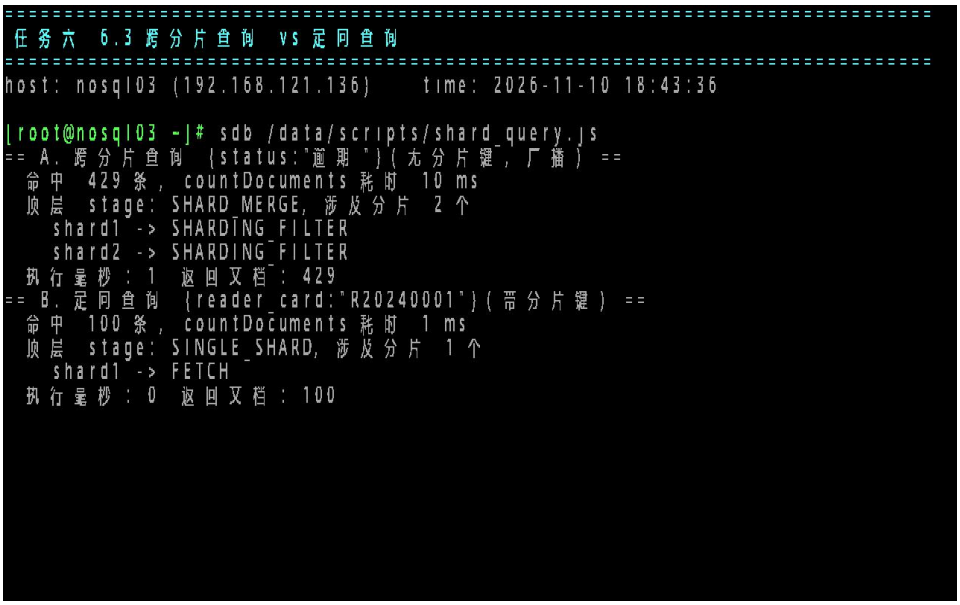


图 6-2 跨分片查询（SHARD_MERGE）与定向查询（SINGLE_SHARD）对比

认证方面，分片集群同样启用 authorization，节点间用与副本集相同的 keyFile（md5 941eba1b6f530cf364026b02d76b9e8e）做内部认证。不带账号访问 mongos 会被拒绝（requires authentication），带 lib_admin 可以正常读取 3000 条流水。由于用户信息统一存放在配置服务器的 admin.system.users 中，只需在 mongos 上创建一次，所有分片共享。

6.6 图形化客户端验收

任务七要求用图形化客户端独立验收。使用 MongoDB Compass 连接 nosql03 上的 mongos（192.168.121.136:27020，账号 lib_admin），数据库页直接列出 library_db 下各集合的文档数与索引数：books 50、readers 30、borrow_logs 3K、category_stats 6、audit_logs 1，且 audit_logs 带 CAPPED 标记，与第 3.4 节把审计日志设计成固定集合的结论一致；这些数字与 4.1 节 countDocuments() 的结果逐项相同。

核对结果为：books 50 / 50、readers 30 / 30、borrow_logs 3K（3000）/ 3000、category_stats 6 / 6、audit_logs 1（CAPPED）/ 1，图形界面与命令行逐项一致；分类聚合的 6 行结果也逐行对齐，仅文学类平均评分因舍入策略不同显示为 4.62 与 4.63。

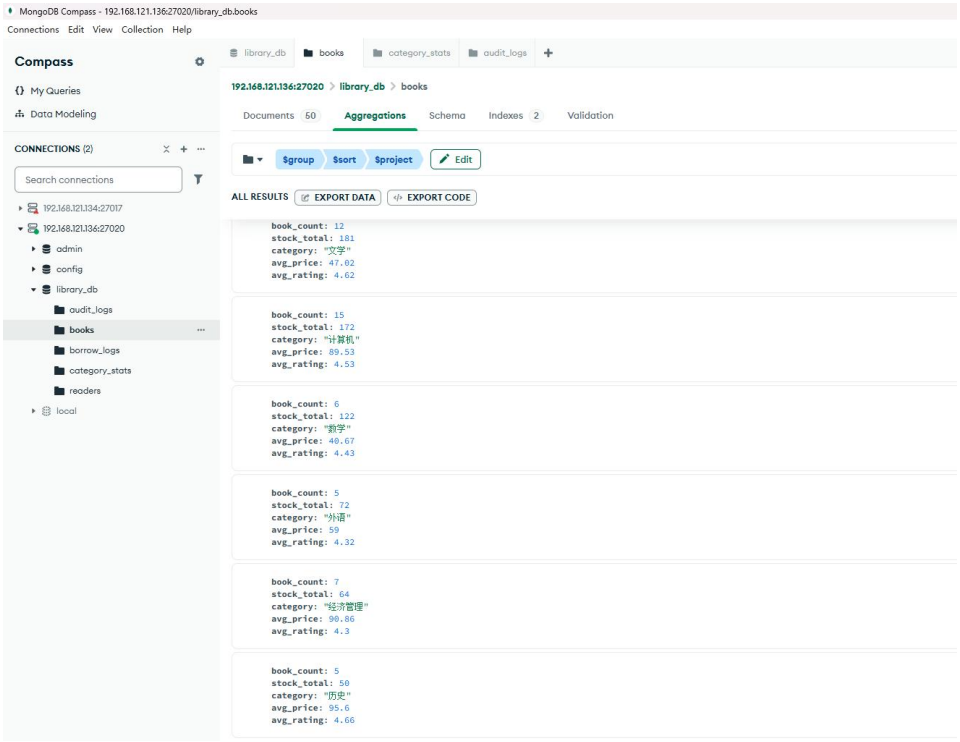


图 6-3 在 Compass 的 Aggregations 界面执行分类统计聚合

聚合管道与命令行脚本完全一致（$group → $sort → $project）。图形界面的 6 行结果与 4.3 节输出逐行对齐，唯一差异是文学类的平均评分：界面显示 4.62，命令行显示 4.63——聚合的 $round 采用银行家舍入（4.625 收敛到 4.62），命令行用 printf 做四舍五入，属于显示策略差异而非计算差异。统计结果另经 mongoexport 导出为 category_stats.json / .csv 随交付物提交，CSV 用电子表格打开后字段与数值同样可核对。

七、问题排查与总结

7.1 典型问题与解决

问题现象	根因	解决方法
mongod 启动失败：Failed to determine whether it is a member of a config server	配置项 storage.journal.enabled 在 7.0 已废弃	删除该配置项，改用默认 WiredTiger 日志
replSetInitiate 报 quorum check failed / Authentication failed	三台各自生成 keyFile，内容不一致	由一台生成后 scp 分发，统一 md5 后重启再初始化
rs.remove 报 should only be run on a writable PRIMARY	客户端固定连 127.0.0.1，恰好连到从节点	连接串改为三主机 + replicaSet=rs0，由驱动自动路由到主节点
ConfigParser 读取连接串报 InterpolationSyntaxError	密码中的 %40 被当成插值语法	改用 RawConfigParser，禁用插值
GridFS 下载报 'GridOut' object has no attribute 'readinto'	pymongo 4.19 的 GridOut 不再提供 readinto	捕获 AttributeError，回退为 256KB 分块 read() 写入
python -m tools.import_data 报 No module named 'tools'	工作目录不在 src 下，包路径不成立	cd /data/app/src 后执行，或在代码中显式调整 sys.path
SELinux 拒绝 mongod 访问自定义数据目录	系统缺少 mongod_var_lib_t 类型，semanage 无法打标签	将 SELinux 设为 permissive 并写入 /etc/selinux/config，同时在文档中记录
mongos 启动后持续报 FailedToSatisfyReadPreference	配置服务器副本集尚未初始化，没有主节点	先 rs.initiate 配置服务器，再启动/重启 mongos
MongoDB Compass 免安装方式启动失败	普通用户会话无管理员令牌，MSI 安装返回 1603；解包出的可执行文件缺运行时依赖，启动即退出（0x80000003）	改用管理员权限正常安装后完成图形化验收（截图 36、37）

表 7-1 实施过程中遇到的问题与处理

7.2 项目总结

本项目以 MongoDB 为核心，完成了从数据建模、索引优化、聚合统计、文件存储，到副本集高可用与分片集群水平扩展的完整链路。在功能层面，六类检索与五项统计全部落地并用截图留存证据；在性能层面，用执行计划与压测数据量化了索引收益，也如实记录了“小数据量下索引反而更慢”的反例；在架构层面，通过读取偏好、故障转移与分片键论证，把 CAP 与 BASE 从概念变成了可验证的配置项与实测数字。

更重要的是工程习惯的沉淀：所有查询与运维动作都脚本化（/data/scripts 下按任务编号组织），所有结论都有截图与命令输出作为证据；连接串、路径、账号全部外置到 config.ini 与环境变量，没有硬编码；每个阶段的部署配置（单机、副本集、配置服务器、分片、路由）都保留在 /etc 下并同步进交付物，便于复现。

后续可改进的方向有三点：其一，接入 Atlas Search 或外部分词服务，解决中文全文检索只能做左前缀匹配的短板；其二，对跨集合的借还操作引入多文档事务，进一步消除“库存已扣、流水未写”的极端窗口；其三，补充监控与容量规划，采集 mongostat / serverStatus 指标并对 chunk 分布做告警，让集群运维从“人工演练”走向“持续可观测”。

附录 A 过程截图索引

过程截图共 36 张，按任务编号存于交付物 04-过程截图 目录，命名即序号 + 任务关键词。

- 任务二：02_doc_types（books 字段与 BSON 类型）
- 任务三：01_import_check（导入计数）、03_deploy（单机部署）、04—09（六类检索）、10—14（五项统计与统计物化）、15—18（索引前后与 30 万压测）、19—20（三角色权限验证）
- 任务四：21_app_menu（12 项菜单）、22_app_search（检索与统计）、23_app_gridfs（上传与字节校验）
- 任务五：25_rs_init（一主两从初始化）、26_rs_readpref（三种读取偏好）、27_rs_failover（切换 3.8 秒）、28_rs_write（切换期写入）、29_rs_addremove（节点增删）、30_rs_auth（认证）
- 任务六：31_shard_topology（拓扑、分布与分片键）、32_shard_query（跨分片 vs 定向）、33_shard_auth（集群认证与 keyFile）
- 任务七：34_s7_accept（分片集群侧数据量、统计集合与导出文件核对）、35_s7_export_view（导出 CSV 在电子表格中的呈现）、36_s7_compass_counts（Compass 核对各集合文档数）、37_s7_compass_aggregation（Compass 聚合界面结果）

附录 B 端口与目录约定

用途	端口	数据目录	配置文件
单机实例	27017	/data/mongodb/single/db	/etc/mongod-single.conf
副本集成员	27017	/data/mongodb/rs/db	/etc/mongod-rs.conf
配置服务器	27019	/data/mongodb/cfg/db	/etc/mongod-cfg.conf
分片节点	27018	/data/mongodb/shard/db	/etc/mongod-shard.conf
路由 mongos	27020	—	/etc/mongos.conf

表 B-1 端口与目录约定

附录 C 运行说明

1) 启动数据库：单机执行 mongod --config /etc/mongod-single.conf；副本集三台分别执行 mongod --config /etc/mongod-rs.conf；分片集群按“配置服务器 → 分片 → mongos”的顺序启动。

2) 导入数据：cd /data/app/src && python3 -m tools.import_data --drop。

3) 运行应用：python3 main.py 进入菜单；或使用子命令，例如 python3 main.py search --keyword 数据、python3 main.py upload --isbn 9787115428028 --path /data/import/gridfs_files/ebook_1.pdf --kind ebook。

4) 连接串集中在 config.ini，切换单机/副本集/分片只需修改 uri 一行，也可用环境变量 MONGODB_URI 覆盖。